```
      expected collection contained:  [{"payee"=>"Starbucks",                ↩
      "amount"=>5.75, "date"=>"2017-06-10", "id"=>42}, {"payee"=>"Zoo",      ↩
      "amount"=>15.25, "date"=>"2017-06-10", "id"=>42}]
      actual collection contained:    []
      the missing elements were:      [{"payee"=>"Starbucks",                ↩
      "amount"=>5.75, "date"=>"2017-06-10", "id"=>42}, {"payee"=>"Zoo",      ↩
      "amount"=>15.25, "date"=>"2017-06-10", "id"=>42}]
```

« *truncated* »

There's not much point in putting off the inevitable. We're going to have to write some code to save and load expenses.

Saving Your Progress: Pending Specs

Before veering off into low-level implementation, it's a good idea to save your work so far. We don't recommend leaving specs in a failing state, though. So let's mark this one as in progress. Add the following highlighted line at the top of your spec:

04-acceptance-specs/06/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb

```
it 'records submitted expenses' do
  pending 'Need to persist expenses'
```

Now, when you run your specs, you will get a reminder of what you were working on:

```
$ bundle exec rspec
*

Pending: (Failures listed here are expected and do not affect your suite's ↩
status)

  1) Expense Tracker API records submitted expenses
     # Need to persist expenses
     Failure/Error: expect(expenses).to contain_exactly(coffee, zoo)

       expected collection contained:  [{"payee"=>"Starbucks",              ↩
       "amount"=>5.75, "date"=>"2017-06-10", "id"=>42}, {"payee"=>"Zoo",    ↩
       "amount"=>15.25, "date"=>"2017-06-10", "id"=>42}]
       actual collection contained:    []
       the missing elements were:      [{"payee"=>"Starbucks",              ↩
       "amount"=>5.75, "date"=>"2017-06-10", "id"=>42}, {"payee"=>"Zoo",    ↩
       "amount"=>15.25, "date"=>"2017-06-10", "id"=>42}]
     # ./spec/acceptance/expense_tracker_api_spec.rb:46:in `block (2        ↩
     levels) in <module:ExpenseTracker>'

Finished in 0.03437 seconds (files took 0.1271 seconds to load)
1 example, 0 failures, 1 pending
```